

Parallel Multisource Social Media Analysis for Xenophobia and Sentiment During the 2026 FIFA World Cup

Samantha Suquilanda

Computer Science Student

ssuquilanda@est.ups.edu.ec

Felipe Peralta

Computer Science Student

dperaltap2@est.ups.edu.ec

José Abad

Computer Science Student

jabadc3@est.ups.edu.ec

Abstract—International football amplifies xenophobic hostility online, where sporting criticism, national identity, and discrimination coexist and hostility is frequently disguised as humor, irony, or coded spelling. This work presents an integrated web platform for collecting and analyzing social-media comments related to xenophobia and sentiment during the 2026 FIFA World Cup. The design separates two computational workloads: I/O-bound acquisition and CPU-bound transformer inference. Four producer threads extract live content from Bluesky, X, YouTube, and Mastodon, while a single consumer normalizes and stores records in SQLite. A persistent `ThreadPoolExecutor` then applies pysentimiento models for three-class sentiment and, when supported, hate-speech detection. The study addresses three questions: whether workload-aware parallelism accelerates a real system, whether a producer-consumer design preserves partial results under source failure, and how sentiment and hate vary across platforms and languages. Four complete live runs produced 354 classified comments. Four-source extraction achieved speedups from 1.26 to 1.37, increasing to 2.71 after excluding the slowest source. On a fixed benchmark of 600 texts, six processes reduced classification time from 359.8 s to 137.9 s, a 2.61 speedup. In a directed corpus of 8,783 comments, 55.7% were negative and 29.9% were marked as hateful. X showed the highest hostility, whereas Bluesky contained substantially more counter-speech, confirming that platform choice reshapes the observed picture. Manual review further revealed that coded hostility—altered spellings such as "m0n0," blended insults such as "mexisimios," and emoji or irony—is systematically under-detected, so the reported hate rate is a lower bound. The results show that workload-aware parallelism improves execution time, but platform diversity and human review of implicit hate remain necessary for valid interpretation.

Index Terms—parallel computing, social media, sentiment analysis, hate speech, xenophobia, natural language processing

I. INTRODUCTION

International football generates large, rapidly changing conversations in which sporting criticism, humor, national identity, and discriminatory hostility coexist. During the 2026 FIFA World Cup, this environment provides a relevant case for studying xenophobia across platforms. The computational challenge is not limited to language classification. Social networks expose different search mechanisms, authentication requirements, data formats, response times, and failure modes, while the resulting text is short, multilingual, informal, and often encoded through irony, spelling variation, emoji, or indirect stereotypes.

Sentiment and hate speech must therefore be treated as related but distinct variables. A negative comment about team performance is not necessarily hateful, and an apparently humorous statement may still target a national or ethnic group. A sentiment-only pipeline would collapse these cases into a single dimension and obscure the phenomenon under study.

This paper presents a query-to-interpretation platform that combines concurrent multisource acquisition, normalized persistence, transformer-based classification, and auditable visualization. Its main contribution is a workload-aware architecture: threads are assigned to I/O-bound extraction, whereas independent processes execute CPU-bound inference. The system also preserves platform provenance, original text, language, search criterion, lexicon indicators, and model outputs so that aggregate results remain inspectable.

The evaluation combines bounded live searches with a larger historical corpus. Live runs validate the integrated application and its parallel behavior. The historical corpus contains 396,841 valid records from five sources, including a directed subset of 8,783 texts selected through a curated xenophobia-related lexicon. The study reports execution time, speedup, failure isolation, platform-level distributions, and qualitative classification limitations. Since no human-annotated gold standard was available, precision, recall, and F1 are not claimed.

The work is organized around three research questions. RQ1 asks whether assigning threads to acquisition and processes to inference produces measurable acceleration under real source latency and model-memory constraints. RQ2 asks whether a producer-consumer design can preserve partial results when one external platform fails. RQ3 asks how sentiment and model-detected hate vary across platforms, languages, and thematic groups, and which cases remain unresolved by automatic classification. These questions connect the software architecture to the substantive analysis instead of treating the application as a collection of independent components.

The evaluation confirms that workload-aware parallelism is effective only when the full stack is controlled: extraction speedup reached 2.71 across three sources, and classification speedup rose from approximately 0.97 to 2.61 once internal-thread oversubscription was eliminated. The substantive analysis further shows that platform choice reshapes the observed

phenomenon and that coded hostility is systematically under-detected. The remainder of this paper is organized as follows. Section II reviews related work on online hate in sport, post-API data access, and social-language transformers. Section III describes the system architecture, data regimes, and evaluation methodology. Section IV reports parallel performance together with live and historical findings. Section V discusses limitations and recommendations, and Section VI concludes.

II. RELATED WORK

Research on online hate and sport identifies social media as a central environment in which racism, sporting identity, and platform governance intersect [1], [2]. Studies of discriminatory football discourse also show that hostility can be expressed through humor, racialization, and indirect cultural references rather than only explicit slurs [3]. Irony-aware abusive-language research further demonstrates why a general classifier may miss camouflaged attacks [4].

Cross-domain resources such as MetaHate illustrate the fragmentation of hate-speech data across platforms, languages, and annotation schemes [5]. This limits the validity of conclusions based on a single source. At the systems level, concurrent crawling research supports separating network-bound downloading from processing [6]. However, current social-media research also operates in a restricted “post-API” environment in which access depends on platform-specific policies and authentication mechanisms [7]. Open protocols such as Bluesky’s AT Protocol provide an alternative research surface, but they are not equivalent to centralized platforms [8].

The selected NLP approach is grounded in social-language transformers. HatEval established a multilingual benchmark for hate directed at immigrants and women [11], while work on latent hatred and emoji-based hate confirms that implicit or symbolic hostility remains difficult for conventional models [9], [10]. The implementation uses *pysentimiento*, which provides sentiment and hate-speech analyzers for social text [12]. Its language routing is consistent with domain-specific models developed for Spanish social media, multilingual Twitter, and English tweets [13]–[15]. In contrast to model-only studies, this work evaluates the complete acquisition, persistence, inference, and interpretation pipeline.

III. SYSTEM AND METHODOLOGY

A. Data Regimes

The live regime accepts a user query and any subset of Bluesky, X, YouTube, and Mastodon. The backend limits extraction to 40 records per network and 20 per search criterion, allowing interactive execution. Bluesky uses AT Protocol text search; X uses an automated browser with a manually authenticated session; YouTube uses Data API v3 search and comment retrieval; and Mastodon maps the query to public hashtag timelines. Source adapters share a common output contract but retain source-specific access logic.

The historical regime supports descriptive analysis. After curation, normalization, and deduplication, it contains 396,841

records: 357,064 from YouTube, 26,122 from Bluesky, 7,651 from Tumblr, 4,846 from X, and 1,158 from TikTok. A directed subset of 8,783 records matched a curated xenophobia-related lexicon. This subset concentrates potentially relevant content but is not treated as a manually confirmed xenophobia corpus.

Both regimes use the same normalized record contract. Each record retains a platform identifier, source network, search strategy, search criterion, text, language, author, publication date, original URL, source-specific metrics, and extraction timestamp. Records initially retrieved through broad search can be promoted to a directed strategy when their text matches the lexicon. This distinction is important because retrieval evidence and semantic prediction answer different questions: a lexicon match explains why a record was selected for closer analysis, whereas the transformer output estimates sentiment or explicit hate. Preserving both fields prevents the directed subset from being misrepresented as ground truth.

TABLE I
EVALUATION DATA REGIMES

Regime	Records	Sources	Purpose
Live runs	354	4	Integration and timing
Historical corpus	396,841	5	Descriptive context
Directed subset	8,783	5	Xenophobia-focused analysis
Classification benchmark	600	–	Process speedup

B. Workload-Aware Architecture

The browser communicates with a FastAPI backend through HTTP and JSON. A search request returns HTTP 202 and an identifier, while the frontend polls state, records, and aggregate results. The API exposes operations for listing supported networks, creating a search, retrieving its state, reading paginated records, and obtaining aggregate results. Pydantic validates a query length between 2 and 200 characters. The frontend and backend do not share implementation code; their explicit contract is the HTTP interface.

SQLite stores the search, normalized records, per-source execution results, and model classifications. Four logical tables separate these concerns: the search table stores query and state; the record table stores source evidence and applies uniqueness by search, network, and external identifier; the sentiment table stores labels and scores independently of the original post; and the per-network result table stores duration, record count, and error. Write-ahead logging permits reads of committed data while the consumer writes. Batched commits avoid the throughput loss of committing every post individually. Separating classification from the source record also preserves evidence when inference is delayed or one block fails.

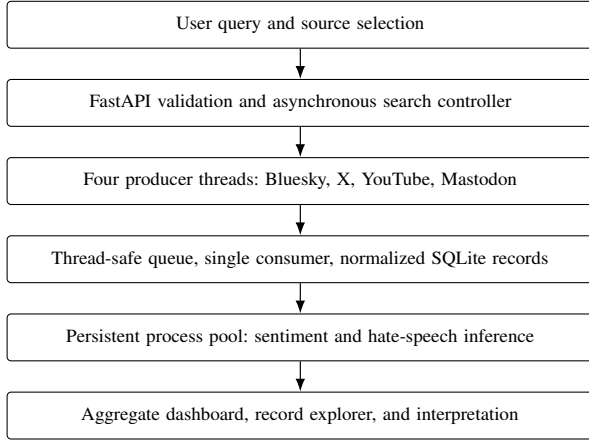


Fig. 1. Workload-aware query-to-interpretation pipeline.

For acquisition, the orchestrator starts one producer thread per selected source. Producers emit normalized records to a thread-safe queue. Queue and always place a unique sentinel when they finish, including after exceptions. A single consumer counts sentinels, performs deduplication, and persists records. This centralizes the short write-critical section without serializing slow network operations. Errors are isolated per source; therefore, an expired X session does not terminate the other producers.

For classification, a persistent `ProcessPoolExecutor` divides records into blocks of 32 texts. Each worker caches its models and returns positive, negative, or neutral sentiment with the maximum model probability. The backend supports sentiment analyzers for Spanish, English, Italian, and Portuguese. Hate-speech analysis is applied for Spanish and English; unsupported languages remain undefined rather than being interpreted as non-hateful. Missing or unsupported sentiment language values are routed to Spanish, a design limitation discussed later. Completed futures are collected independently, so a failed block does not invalidate classifications returned by other workers.

The frontend maps these outputs into four views. The live-search view submits the query and reports source-level execution evidence; the dashboard displays global and per-platform distributions; the explorer exposes every comment with platform, language, sentiment, score, and hate value; and the interpretation view combines the current bounded sample with versioned historical findings. Progress is inferred through polling rather than WebSockets or server-sent events. This interface is analytically relevant because users can inspect the original records behind aggregate charts instead of accepting model totals as unexplained conclusions.

The adapters deliberately standardize output rather than access. Bluesky supports text search through the AT Protocol, while YouTube first identifies videos and then retrieves comments. X depends on a manually authenticated browser session, making session expiration and captchas explicit failure conditions. Mastodon exposes public hashtag timelines, so a phrase containing spaces does not behave like a full-text

query and may return no records unless it corresponds to an established hashtag. Tumblr and TikTok remain available only in the historical corpus. Keeping this distinction visible avoids presenting fixed-tag or semi-manual collection as equivalent to live query execution.

Data integrity is maintained at three levels. First, uniqueness is scoped to search identifier, network, and external identifier, which removes duplicates inside a run without preventing the same public post from appearing in independent searches. Second, the original text and metadata remain unchanged when a classification is added. Third, aggregate queries use left joins so that incomplete or failed inference remains observable. The system can therefore distinguish a record classified as non-hateful from a record for which no supported hate model was executed.

Nested parallelism required explicit control. PyTorch and numerical libraries can create internal threads inside every worker process. The final implementation sets `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `NUMEXPR_NUM_THREADS` to one and calls `torch.set_num_threads(1)`. This prevents process-level parallelism from being undermined by oversubscription.

C. Evaluation

Four complete live searches enabled all four networks with a maximum of 40 records per source. The sequential extraction reference was the sum of producer durations, while concurrent wall time was bounded by the slowest overlapping source. Extraction speedup was computed as

$$S_{ext} = \frac{\sum_{i=1}^n T_i}{T_{wall}}. \quad (1)$$

A controlled test repeated extraction without X to separate orchestration behavior from source-latency heterogeneity. Failure isolation was tested by removing the X session. Classification used a fixed 600-text sample with one and six processes; efficiency was calculated as speedup divided by process count.

The four live runs were aggregated only to demonstrate functional behavior because each search was bounded and query-dependent. Descriptive conclusions instead use the directed historical subset and are reported by platform, language, and thematic axis. Classification validity was assessed qualitatively through inspection of counter-speech and coded hostility. This procedure can reveal systematic errors but cannot estimate general model accuracy; consequently, stored model probabilities are treated as confidence values rather than empirical correctness.

IV. RESULTS

A. Parallel Performance

Across four complete runs, X averaged 14.15 s, compared with 1.85 s for YouTube, 1.44 s for Bluesky, and 1.13 s for Mastodon. Consequently, four-source extraction achieved only 1.26–1.37 speedup because wall time approached X latency. When X was excluded, speedup increased to 2.71 in the best controlled run, close to the theoretical maximum

of three sources. This confirms that the moderate four-source acceleration was caused primarily by source heterogeneity rather than serialized orchestration.

TABLE II
FOUR COMPLETE LIVE EXTRACTION RUNS

Query	Wall	Sum	Speedup	End-to-end
Racismo futbol	14.31 s	19.60 s	1.37	26.8 s
Mundial 2026 racismo	13.01 s	17.06 s	1.31	184.9 s
Mundial2026	14.73 s	19.29 s	1.31	34.0 s
Mundial2026 repeat	14.55 s	18.31 s	1.26	32.7 s

The 184.9 s end-to-end result occurred during lazy loading of additional language and hate-speech models and is therefore a warm-up outlier rather than stable operating time. Subsequent complete searches required approximately 33–34 s. The distinction between source extraction time and complete request time is important: thread concurrency reduces acquisition latency, but model initialization and inference remain separate costs.

The 600-text benchmark required 359.8 s with one process and 137.9 s with six, yielding a 2.61 speedup and 43% efficiency. Before internal-thread limits were applied, speedup was approximately 0.97. The improvement after pinning demonstrates that process count alone is not a sufficient performance strategy; nested threading, model initialization, interprocess communication, and memory capacity must also be controlled. The result is consistent with the limits imposed by serial overhead and resource contention [16].

TABLE III
PARALLEL PERFORMANCE SUMMARY

Experiment	Sequential	Parallel	Speedup
Four-source extraction, best run	19.60 s	14.31 s	1.37
Four-source extraction, lowest	18.31 s	14.55 s	1.26
Three-source extraction, best	3.71 s	1.37 s	2.71
Classification, 600 texts	359.8 s	137.9 s	2.61

The failure-isolation test removed the X browser session. YouTube returned 40 records in 1.26 s, Bluesky 19 in 1.04 s, and Mastodon 20 in 0.77 s. X returned no records and a session error, but the search finished successfully with 79 classified comments. Partial success remained visible rather than being confused with complete execution.

B. Live and Historical Findings

The four live searches produced 354 comments. YouTube and X had the highest negative proportions, at 53.1% and 52.5%, respectively. Bluesky was predominantly neutral at 54.1%, while Mastodon reached 85.0% neutral. The query “racismo futbol” produced 79 comments, of which 75.9% were negative and 12.7% were marked as hateful. These bounded samples validate the workflow but are not treated as population estimates because results depend on query wording, platform availability, and source-specific search behavior.

TABLE IV
AGGREGATE RESULTS FROM FOUR LIVE SEARCHES

Platform	<i>n</i>	Negative	Neutral	Hate
YouTube	160	53.1%	23.8%	10.0%
X	80	52.5%	42.5%	5.0%
Bluesky	74	35.1%	54.1%	4.1%
Mastodon	40	15.0%	85.0%	5.0%

The directed historical subset provides the main descriptive evidence. Globally, 55.7% of its 8,783 comments were negative, 33.7% neutral, and 10.6% positive; 29.9% were marked as hateful. X showed the highest negativity and hate-speech incidence. YouTube had a similar hate-speech proportion but a less negative overall distribution. Bluesky differed substantially, with 50.5% neutral content and only 5.2% model-detected hate.

TABLE V
DIRECTED CORPUS RESULTS BY PLATFORM

Platform	<i>n</i>	Neg.	Neu.	Hate
X	4,739	70.1%	22.7%	40.3%
YouTube	832	52.8%	32.2%	39.7%
Tumblr	41	58.5%	24.4%	15.8%
Bluesky	3,161	35.0%	50.5%	5.2%
TikTok	10	10.0%	80.0%	0.0%

Manual inspection explains part of this contrast. Bluesky frequently contained reports or condemnations of racism, meaning that lexicon terms appeared in counter-speech rather than attacks. This demonstrates why lexicon matching is useful for directed retrieval but cannot serve as a final semantic label. Platform-level comparison is therefore essential: a study limited to X could overrepresent direct aggression, whereas one limited to Bluesky could overrepresent denunciation and meta-conversation.

Language and thematic distributions further showed that volume and virulence are different. Spanish records had 75.4% negative sentiment, compared with 43.3% for English and 40.0% for Portuguese, although platform composition may contribute to the association. The anti-Black or simian axis contained 4,765 records but 21.7% model-detected hate, while the smaller colonial/authenticity and anti-Mexican/migrant axes exceeded 56% hate. The most frequent theme was therefore not the most explicitly virulent.

TABLE VI
LANGUAGE AND THEMATIC CONTRASTS

Group	Records	Main proportion
Spanish	3,654	75.4% negative
English	3,947	43.3% negative
Portuguese	940	40.0% negative
Anti-Black/simian	4,765	21.7% hate
Colonial/authenticity	211	57.5% hate
Anti-Mexican/migrant	893	56.6% hate

C. Interpretive Implications

The architecture and the substantive findings are directly connected. A common record contract makes different platforms

interchangeable at the pipeline boundary, but the reported distributions still retain their source provenance. This is essential because the platforms do not provide equivalent populations or access mechanisms. X and YouTube concentrated more direct aggression in the directed corpus, whereas Bluesky captured a larger amount of counter-speech and discussion about racism. A single-source study would therefore describe the same event differently depending on the selected platform.

The performance results also clarify the limits of parallelism in an applied system. Starting all producers together prevents extraction time from becoming the sum of all source times, yet the slowest source remains the wall-clock bound. Similarly, process-based inference can exploit independent text blocks, but model initialization, process communication, memory consumption, and nested numerical threads prevent linear scaling. The measured speedups are therefore evidence of correct workload assignment rather than claims of ideal acceleration.

Finally, the dual-axis analysis is not an optional interface feature. Negative sentiment can express disappointment, criticism, or rivalry without targeting a national or ethnic group. Conversely, coded humor can convey xenophobic meaning without receiving a strongly negative sentiment score. Preserving sentiment, hate prediction, lexicon evidence, language, platform, and original text allows these cases to be compared instead of collapsing them into a single label.

V. DISCUSSION AND LIMITATIONS

The results support three conclusions about the system design. First, threads and processes are not interchangeable optimizations: each is effective only when aligned with the corresponding I/O- or CPU-bound workload. Second, the common record contract and single-consumer persistence model allow heterogeneous platforms to share one analytical pipeline while preserving failure isolation. Third, sentiment and hate speech must remain separate analytical axes because negativity does not imply xenophobia and indirect xenophobia may not appear strongly negative.

The semantic results must nevertheless be interpreted cautiously. Manual review found altered spellings such as “m0n0” and “macac0s,” blended insults such as “mexisimios,” “Ecuakong,” and “Mechico,” symbolic substitutions, and ironic statements that were negative but not marked as hateful. This pattern agrees with prior findings on implicit, emoji-based, and irony-mediated abuse [4], [9], [10]. Accordingly, the observed 29.9% hate-speech rate is a lower bound on explicit model-detected hate within the lexicon-directed subset, not the true prevalence of xenophobia.

Counter-speech presents the opposite ambiguity. A post may repeat a racist expression in order to report, criticize, or condemn it. Lexicon retrieval appropriately keeps such material available for review, while the independent hate label can distinguish some denunciations from attacks. Quoted abuse, sarcasm, and insufficient context can still confuse the classifier. For this reason, the system’s analytical value lies not only in

automatic totals but also in preserving the original text and provenance needed to audit those totals.

The study has seven principal limitations. No human-annotated gold standard was available, so model probabilities cannot be converted into accuracy, precision, recall, or F1. The directed subset is lexicon-selected and cannot represent all World Cup conversation. In the live runs, 193 of 354 records lacked language metadata, and routing missing values to Spanish may introduce mismatch. Platform access is unstable: X depends on an authenticated browser session, and Mastodon hashtag timelines do not provide the same recall as full-text search. Tumblr and TikTok are represented historically but not as live sources. Each inference process loads approximately 3.5–4 GB of models, limiting production worker count despite the six-process benchmark. The extraction speedup ceiling is dominated by X latency, and the frontend reports phase-level progress through polling rather than granular streaming events.

These limitations do not invalidate the functional and performance results, but they define the scope of interpretation. The architecture demonstrates reproducible execution and evidence organization; it does not establish an epidemiological prevalence of xenophobia. The historical percentages describe a lexicon-directed subset, and the live percentages describe bounded searches executed through unequal source interfaces. Distinguishing those regimes is necessary to avoid converting engineering evidence into unsupported population claims.

Future work should create a multi-annotator validation subset covering explicit hate, implicit hate, counter-speech, irony, quoted abuse, and ambiguous sporting criticism. At least two reviewers should label the same material, disagreements should be resolved systematically, and inter-annotator agreement, precision, recall, and F1 should be reported. The classifier should also combine leetspeak normalization, orthographic variants, emoji-aware representations, irony-sensitive models, and targeted human review. Source adapters should remain replaceable because access policies can change independently of the application.

A production deployment would additionally benefit from explicit progress events and workload-aware resource selection. Server-sent events or WebSockets could expose source completion and block-level inference progress, while the backend could select worker count according to available RAM, loaded languages, and batch size. These changes would improve usability and resource safety without altering the fundamental two-stage parallel design.

A. Recommendations

The first priority is a human-annotated validation subset. Clear guidelines should distinguish sentiment, explicit hate, implicit hate, counter-speech, irony, quoted abuse, and ordinary sporting criticism. At least two independent annotators should review the same records, with disagreements resolved systematically. This would permit precision, recall, F1, and inter-annotator agreement to replace the current qualitative assessment.

The second priority is explicit treatment of coded language. The existing lexicon and provenance fields provide an appropriate base for a hybrid classifier that incorporates leetspeak normalization, orthographic variants, emoji-aware representations, and irony-sensitive analysis. Automatic screening should continue to be paired with targeted human inspection because no single transformation resolves all contextual ambiguity.

The third priority is operational resilience. Source adapters should remain independent and replaceable because authentication, quotas, search interfaces, and platform policy can change without warning. The existing common contract allows an unavailable source to be replaced without redesigning persistence, inference, or visualization. A future deployment should also record adapter versions so that changes in access behavior remain traceable across experiments.

The fourth priority is workload-aware deployment. Server-side events or WebSockets could expose granular source and inference progress, while a resource controller could select the number of workers according to available RAM, loaded languages, and batch size. The production default of two workers is appropriate for the current memory footprint even though six processes were useful for the controlled benchmark.

VI. CONCLUSION

The proposed platform integrates live multisource extraction, normalized storage, transformer inference, and auditable visualization in one workflow. Its principal technical result is that workload-aware parallelism improves performance only when the entire execution stack is controlled: extraction benefited from producer threads, classification from processes, and inference speedup rose from approximately 0.97 to 2.61 after preventing internal-thread oversubscription. Its principal analytical result is that platform choice changes the observed picture of xenophobia. X concentrated direct hostility, while Bluesky contained substantially more counter-speech. The dual-axis design also confirmed that sentiment alone is insufficient for analyzing xenophobia.

Four contributions summarize the work. First, the common adapter contract integrates unequal access mechanisms without hiding their source-specific limitations. Second, producer-level failure isolation allows partial searches to complete and remain auditable. Third, the combination of lexicon-directed retrieval with independent sentiment and hate labels preserves the distinction between selection and interpretation. Fourth, the evaluation connects implementation decisions with measured behavior, including the source-latency ceiling and the effect of internal-thread oversubscription.

The platform therefore provides an efficient screening and evidence-organization system rather than an autonomous measurement of xenophobia prevalence. Its results support

comparative exploration, identify records for review, and demonstrate the performance benefits of a two-stage parallel architecture. Valid substantive interpretation still requires platform-aware sampling, transparent treatment of missing language information, and human review of coded or context-dependent language.

REFERENCES

- [1] C. Kearns, G. Sinclair, J. Black, M. Doidge, T. Fletcher, D. Kilvington, K. Liston, T. Lynn, and P. Rosati, "A Scoping Review of Research on Online Hate and Sport," *Communication & Sport*, vol. 11, no. 2, pp. 402–430, 2023.
- [2] K. Hylton, D. Kilvington, J. Long, A. Bond, and I. Chaudry, "The Challenge of Social Media and Platformed Racism in the English Premier League and Football League," *International Review for the Sociology of Sport*, vol. 59, no. 6, pp. 844–867, 2024.
- [3] E. Glynn, D. H. K. Brown, and L. Edwards, "Discriminatory Meme Culture on Football Twitter: Othering and Racialisation Through Insensitive Humour," *Media Watch*, vol. 16, no. 2, pp. 138–169, 2025.
- [4] S. Frenda, V. Patti, and P. Rosso, "When Sarcasm Hurts: Irony-Aware Models for Abusive Language Detection," in *Experimental IR Meets Multilinguality, Multimodality, and Interaction*, vol. 14163, 2023.
- [5] P. Piot, P. Martín-Rodilla, and J. Parapar, "MetaHate: A Dataset for Unifying Efforts on Hate Speech Detection," in *Proc. ICWSM*, 2024.
- [6] W. Jiang, "A Novel Multi-Threaded Web Crawling Model," arXiv:2407.10440, 2024.
- [7] D. Freelon, C. Monzer, G. Jeon, C. Moy, and N. Williams, "The Post-API Age of Social Media Data Access: Past, Present, and Future," *The ANNALS of the American Academy of Political and Social Science*, vol. 715, no. 1, pp. 16–37, 2024.
- [8] M. Kleppmann *et al.*, "Bluesky and the AT Protocol: Usable Decentralized Social Media," in *Proc. ACM CoNEXT Workshop on the Decentralization of the Internet*, 2024.
- [9] M. ElSherief *et al.*, "Latent Hatred: A Benchmark for Understanding Implicit Hate Speech," in *Proc. EMNLP*, pp. 345–363, 2021.
- [10] H. R. Kirk, B. Vidgen, P. Röttger, T. Thrush, and S. A. Hale, "Hatemoji: A Test Suite and Adversarially-Generated Dataset for Benchmarking and Detecting Emoji-Based Hate," in *Proc. NAACL-HLT*, 2022.
- [11] V. Basile *et al.*, "SemEval-2019 Task 5: Multilingual Detection of Hate Speech Against Immigrants and Women in Twitter," in *Proc. SemEval*, pp. 54–63, 2019.
- [12] J. M. Pérez *et al.*, "pysentimiento: A Python Toolkit for Opinion Mining and Social NLP Tasks," arXiv:2106.09462, 2021.
- [13] J. M. Pérez, D. A. Furman, L. Alonso Alemany, and F. Luque, "RoBERTuito: A Pre-Trained Language Model for Social Media Text in Spanish," in *Proc. LREC*, 2022.
- [14] F. Barbieri, L. Espinosa-Anke, and J. Camacho-Collados, "XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond," in *Proc. LREC*, pp. 258–266, 2022.
- [15] D. Q. Nguyen, T. Vu, and A. T. Nguyen, "BERTweet: A Pre-Trained Language Model for English Tweets," in *Proc. EMNLP System Demonstrations*, pp. 9–14, 2020.
- [16] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, pp. 483–485, 1967.
- [17] K. Mimizuka, M. A. Brown, K.-C. Yang, and J. Lukito, "Post-Post-API Age: Studying Digital Platforms in Scant Data Access Times," arXiv:2505.09877, 2025.
- [18] M. Lisker and H. Mihaljević, "Data Ethics in the Fediverse: Analyzing the Role of Instance Policies in Mastodon Research," in *Next-Gen and Alternative Social Media Workshop, ICWSM*, 2025.
- [19] J. Cañete, G. Chaperon, R. Fuentes, J.-H. Ho, H. Kang, and J. Pérez, "Spanish Pre-Trained BERT Model and Evaluation Data," in *Practical ML for Developing Countries Workshop, ICLR*, 2020.